

[BLUE PULSE](#) ► [DOCS](#)

Propuesta · Integrar el TFG (SSI sobre Ethereum local) con VulnFlow

Fusión TFG (SSI Ethereum local) × VulnFlow

[Python](#)[FastAPI](#)[PostgreSQL 16 / SQLite](#)[Ed25519](#)[Jinja2](#)

Propuesta · Integrar el TFG (SSI sobre Ethereum local) con VulnFlow

Resumen en una frase: usar la infraestructura de identidad soberana del TFG (Identity/Revocation/AccessPolicy + firma fuera de cadena) para que VulnFlow deje de firmar con una clave común y pase a **atribuir cada ejecución, hallazgo y informe a un auditor concreto e identificable**, con un **sello de auditoría verificable** y un **RBAC real** sobre qué puede ejecutar cada usuario.

0. Por qué tiene sentido juntarlos

Los dos proyectos comparten cimientos que hacen la integración **barata**:

Cimiento común	TFG (<code>auth-blockchain-tfg</code>)	VulnFlow
Agente Python / FastAPI	proxy <code>:8001</code>	agente <code>:8002</code>
Panel web dark, offline	Block-Auth	Panel VulnFlow
Tipografías y estética	Inter + JetBrains Mono vendidas	idem
Persistencia	PostgreSQL 16 <code>:5433</code>	PostgreSQL 16 <code>:5434</code> (con fallback SQLite)
Criptografía aplicada	firma EIP-191/ed25519, keystore WebCrypto	firma Ed25519 de reportes
Jerarquía de datos	DID → roles → perms sobre tablas	target → runs → findings → evidence

Y **se complementan**:

- El TFG resuelve exactamente la carencia débil de VulnFlow: hoy VulnFlow firma con **una sola clave local compartida** (`data/keys/signer_ed25519.key`). No distingue quién autorizó y ejecutó cada cosa, ni permite revocar a un analista.
- VulnFlow resuelve también una demanda del TFG: su memoria habla de "traza, audit trail y documentación", pero la consola SQL firmada es solo consulta. Un motor de auditoría como VulnFlow **sí genera documentación en vivo**.

Un uso inmediato y coherente: VulnFlow puede **auditar el propio despliegue del TFG** (los puertos 8001/8545/5433 de la máquina) y generar el primer informe técnico con evidencia real de su stack. Es el "demo de la integración" cero-coste.

1. Tres niveles de integración (de menor a mayor ambición)

Se proponen **3 escenarios escalonados**, independientes y acumulativos. Cada uno aporta un capítulo nuevo a la memoria del TFG y un extra real a VulnFlow.

Nivel A · Identidad del auditor (firma con DID)

Objetivo: que cada hallazgo/informe se firme con la **clave personal del auditor** (cifrada por su contraseña, igual que el keystore del TFG), no con una clave común.

Impacto:

- `core/signer.py` mantiene Ed25519 como opción "sin TFG", pero en modo integrado usa el **keystore del auditor** del TFG (`pbkdf2-sha256 + aes-256-gcm`, compatible WebCrypto). El sello del informe diría "auditor did:web:tfg:cardi" con la clave pública derivada, no "signer_ed25519.pub".
- La **evidencia en crudo** queda vinculada al DID que la marcó: atribución inconfundible en el informe.

Material para la memoria:

- Transición de "clave de herramienta" → "clave de persona".
- El keystore cifrado como secret store portable (mismo formato en navegador y backend).

Nivel B · RBAC: ¿quién puede ejecutar qué? (control de acceso real)

Objetivo: el panel de VulnFlow pide **login contra el proxy del TFG** (`:8001`, `POST /auth/verify` + JWT) y los roles del `AccessPolicyRegistry` deciden permisos.

Mapeo propuesto (roles ya existentes en el TFG: `ROLE_ADMIN`/`WRITER`/`VIEWER`/`ANALYST`):

Acción de VulnFlow	Rol mínimo
Registrar / borrar activos	<code>ROLE_ADMIN</code>
Ejecutar módulos <code>reconocimiento</code> (nmap/dig)	<code>ROLE_ANALYST</code>
Ejecutar módulos <code>capa web</code> (gobuster/ffuf/curl)	<code>ROLE_WRITER</code>
Ejecutar el módulo <code>raw</code> (msfconsole)	<code>ROLE_ADMIN</code> (y <code>VULNFLOW_ALLOW_RAW=1</code>)
Marcar hallazgos / remediaciones	<code>ROLE_WRITER</code>
Generar y firmar reportes	<code>ROLE_ADMIN</code> / <code>ROLE_ANALYST</code>

Implementación mínima: un middleware en `api/main.py` que valida el JWT emitido por el TFG y consulta `GET /identities/{did}/role` (o el `can(didHash, tabla, op)` ya existente) antes de ejecutar el módulo. La **revocación se propaga al instante** (la cadena es la fuente de verdad) — exactamente el valor del TFG.

Material para la memoria:

- RBAC basado en blockchain aplicado a un caso real de herramienta ofensiva.
- Revocación en caliente demostrada (un admin quita el acceso y el siguiente `POST /scans` se deniega).

Nivel C · Cadena de custodia inmutable (sello de auditoría `on-chain`)

Objetivo: anclar cada ejecución en la cadena para tener un **libro de auditoría a prueba de manipulaciones** (que nadie borre/altere el historial).

Diseño propuesto:

- Nuevo contrato `AuditRegistry` (Solidity, en el repo del TFG) con un evento/array: `sealRun(didHash, toolRunHash, timestamp)` donde `toolRunHash = keccak256(command || sha256(output) || target)`.
- VulnFlow no necesita web3 directo: llama a un endpoint nuevo del proxy del TFG (p. ej. `POST /audit/seal`), que firma la petición por petición igual que `/q`.
- El informe termina con una sección **"Sellos de auditoría"**: tx/event hash de cada

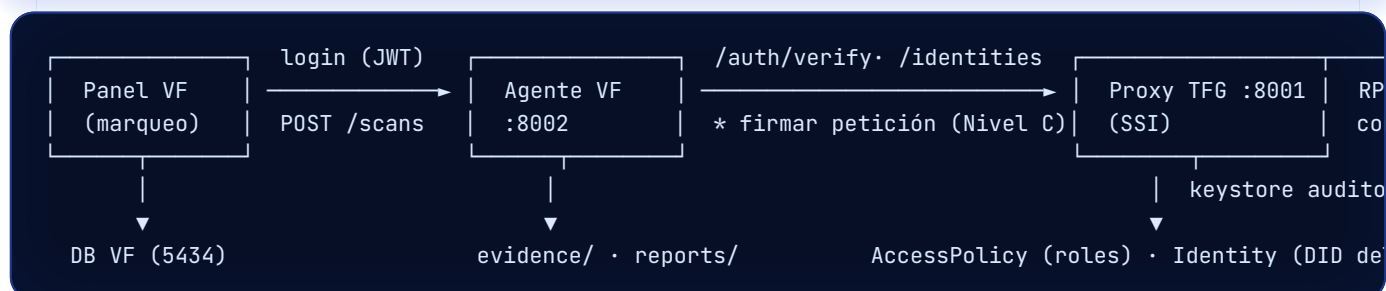
run. Verificar el libro = consultar la cadena (off-chain, sin gas).

- El **ciclo de vida se completa**: la cadena efímera del TFG se reinicia en cada `start` ; los sellos del libro actual quedan ligados al despliegue vigente (y a `scripts/seed.ts` / `provision-cardi.mjs`). Ese matiz de "despliegue + libro" puede ser precisamente el **capítulo de arquitectura** de la memoria.

Material para la memoria:

- Cadena de custodia para evidencia de pentesting: novedoso en el ámbito académico.
- Comparativa propiedad "immutable vs editable" entre historial SQL y sellos on-chain.

2. Arquitectura de la integración (lo que se propone construir)



- **Dos repos, no uno**: el TFG y VulnFlow siguen siendo proyectos separados. La integración es por **HTTP interno entre agentes** (`VulnFlow` → `proxy TFG :8001`) más un **client small** `core/tfg.py` en VulnFlow (variables `TFG_PROXY_URL` , `TFG_JWT` , ...). Cero acoplamiento de despliegue: si el proxy del TFG no responde, VulnFlow vuelve al modo "una clave local" y sigue funcionando (degradación, igual que con SQLite).
- El JWT del TFG se obtiene en el **login del panel de VulnFlow** (el navegador llama al proxy del TFG y VulnFlow guarda el token en sesión) — o, para demo headless, un token de servicio emitido con el keystore de CARDI.

3. Cambios concretos por repo

3.1 En VulnFlow (este repo)

Fichero	Cambio
<code>requirements.txt</code>	+ <code>python-jose</code> (o <code>PyJWT</code>) para validar JWT del TFG — opcional
<code>core/config.py</code>	<code>TFG_PROXY_URL</code> (<code>http://127.0.0.1:8001</code> por defecto), <code>TFG_MODE=off a b c</code>
<code>core/tfg.py</code> (nuevo)	client SSI: <code>login()</code> , <code>role_of(did)</code> , <code>can(did, tabla, op)</code> , <code>seal_run(run)</code> — todo con degradación a no-SSI si <code>TFG_PROXY_URL</code> no responde
<code>core/signer.py</code>	en modo A: firmar con keystore del auditor (derivar clave con la contraseña de sesión) en vez de la clave común
<code>core/models.py</code>	<code>ToolRun.auditor_did</code> , <code>Finding.added_by_did</code> (nulos = modo sin TFG)
<code>api/main.py</code>	middleware opcional de login/JWT (Nivel B)

Fichero	Cambio
<code>api/routes/reports.py</code>	en modo C: al firmar, añadir sección "Sellos de auditoría" en el <code>.md</code>
<code>ui/</code>	pequeño login (solo si <code>TFG_MODE</code> lo activa) + columna "Auditor" en historial
<code>docs/</code>	este documento + nota en el README

3.2 En el TFG (`auth-blockchain-tfg`)

Fichero	Cambio
<code>contracts/AuditRegistry.sol</code> (nuevo) — Nivel C	<code>sealRun(didHash, runHash, ts)</code> (event + almacenamiento), owner del registro = el admin de AccessStack
<code>ignition/modules/AccessStack.ts</code>	desplegar el 4º contrato
<code>scripts/seed.ts</code>	seed del registro + ejemplo de sellos
<code>proxy/main.py</code>	<code>POST /audit/seal</code> (firma la petición como <code>/q</code> , llama a <code>AuditRegistry</code>), y exponer <code>GET /identities/{did}/role</code> ya existente vía pública
<code>test/</code>	tests Mocha+Chai del nuevo contrato (cobertura sigue al 100%)
<code>docs/</code>	DEC nueva (decisión de diseño) + actualizar <code>informe-tfg</code>

4. Roadmap sugerido (orden de trabajo, verificable en cada paso)

Fase	Qué	Cómo se verifica
F0	VulnFlow audita el despliegue del TFG (nmap <code>127.0.0.1 -p 8001,8545,5433</code>)	informe generado con evidencia real
A1	<code>core/tfg.py</code> + login demo contra el proxy del TFG	<code>POST /auth/verify</code> OK desde CLI; <code>role_of(cardi)=ROLE_VIEWER</code>
A2	Firmar hallazgo/informe con DID del auditor	<code>.sig</code> con clave pública del DID; <code>verify</code> OK
B1	Middleware de roles en VulnFlow	vista readonly para VIEWER (sin <code>POST /scans</code>); revocación pasa a 403 al instante
B2	Tests E2E de la matriz de acciones×roles	tabla de casos que pasa/deniega (documentado)
C1	<code>AuditRegistry</code> + endpoint <code>/audit/seal</code>	tx event con <code>runHash</code> ; <code>toolRunHash</code> reproducible con <code>keccak256</code>
C2	Sección "Sellos" en el informe + verificación desde la cadena	informe final con sellos; script <code>verify</code> que comprueba los <code>runHash</code>
C3	Memoria: nueva sección de integración y capturas	PDF del informe + capítulos A/B/C actualizados

Esfuerzo estimado: F0–A2 ~1 jornada (la mayoría es el client HTTP + keystore), B ~1 jornada (middleware + tests), C ~2–3 jornadas (contrato + endpoint + integración).
Total ~una semana real de trabajo de TFG, desglosable en fases entregables.

5. Riesgos y decisiones a tomar

Riesgo / decisión	Conversación abierta
Cadena efímera (<code>--reset</code> en cada <code>start</code>)	Los sellos del libro solo son válidos para el despliegue vigente. Propuesta: anclar un "registro-raíz" a una dirección fija y documentarlo; o persistir un snapshot de direcciones al arrancar.
JWT vs firma por petición	El TFG ya firma cada <code>/q</code> con la clave del navegador (W3C). Para VulnFlow podríamos pedir una firma por <code>POST /scans</code> (más fiel al SSI) o un JWT de sesión (más cómodo). Decisión a tomar en el diseño de detalle (la propuesta asume JWT para B y petición firmada para C).
Un solo despliegue vs dos	Behinder a dos repos comunicados por HTTP. Si el tiempo del TFG manda, Nivel A es suficiente y ya es una contribución; B y C son mejoras incrementalmente entregables.
Keystore en backend (A)	Para demo headless del Nivel A hace falta derivar la clave del usuario con su contraseña en Python (el TFG la deriva en el navegador). Reutilizar el mismo formato (pbkdf2 + aes-gcm, WebCrypto-compatible) para no bifurcar formatos.
Lenguaje del commit	Ambos repos en castellano; si esta integración va a la memoria, mejor documentar también en <code>docs/01-decisiones.md</code> del TFG.

6. Resultado: qué gana cada lado

VulnFlow gana

- Deja de ser "firma de una clave local" → **identidad y atribución** por auditor.
- **RBAC y revocación en caliente** sobre el ejecutable (algo que casi ninguna herramienta ofensiva de laboratorio tiene).
- **Libro de auditoría inmutable** (cadena de custodia) si se llega al Nivel C.

El TFG gana

- Un **caso de uso real y demostrable** de la infraestructura SSI: no es solo un login y una consola, es el control de una herramienta de seguridad completa.
- Material académico: capítulo de "SSI applied to offensive-security tooling" con arquitectura (2 agentes), RBAC, sellos on-chain y capturas reales.
- Una demo de **cero gas y sin internet** que conecta directamente con el estado del arte de identidad soberana aplicada.

El conjunto gana: un laboratorio donde "quién ejecutó qué, cuándo, con qué rol y con qué evidencia" queda **probado criptográficamente**, y el informe de auditoría es a la vez entregable de seguridad y prueba de concepto del TFG.